Modèles algorithmiques fondamentaux sur structures à accès direct

Thème principal : Initiation à l'algorithmique et patrons de conception (Design Patterns) sur structures indicées

Synthèse exécutive : Ce cours pose les fondements méthodologiques indispensables de la conception algorithmique en BUT 1 Informatique. Avant toute écriture de code, l'informaticien doit analyser le contrat d'interface et le mécanisme d'accès de sa structure de données (Direct, Séquentiel ou Dynamique). Le cours formalise trois grands modèles algorithmiques canoniques réutilisables sur des structures à accès direct (tableaux, chaînes de caractères) : (1) Le parcours complet avec traitement systématique, (2) Le parcours complet avec traitement conditionné, et (3) La recherche de première occurrence (avec ses deux variantes). Une rigueur absolue est exigée sur le respect des bornes d'indices (de 0 à N-1 pour éviter les erreurs de dépassement 'out of bounds'), sur l'adoption du principe du pessimisme algorithmique, ainsi que sur l'obligation d'intégrer une double condition d'arrêt pour tout algorithme de recherche.

1. Typologie et Analyse Préalable des Mécanismes d'Accès aux Données

L'étape zéro de toute démarche d'analyse algorithmique consiste à s'interroger sur la nature des accès autorisés par la structure de données sous-jacente. Si l'on manipule une structure à accès direct (un tableau statique de N éléments ou une chaîne de caractères), l'adresse mémoire de n'importe quel élément situé à l'indice 'i' est calculable immédiatement. En revanche, sur un flux séquentiel (tel qu'un fichier CSV lu ligne par ligne), l'ordinateur ne propose que trois opérations fondamentales : (1) se positionner au début du flux, (2) lire l'élément suivant, et (3) tester si la fin du flux est atteinte. Il est physiquement et logiquement impossible de sauter directement à la 50ème ligne sans avoir lu les 49 premières. Appliquer une notation par indice de type T[i] sur un fichier séquentiel constitue donc un contre-sens algorithmique qui invalide entièrement la solution proposée.

- Structure de données : collection regroupant des éléments de même type.
- Accès direct : capacité d'atteindre instantanément n'importe quel élément via un indice ou une clé (ex: T[i]) en temps constant O(1).
- Accès séquentiel : contrainte imposant de démarrer au début de la structure et de progresser pas à pas (début, suivant, fin).
- Accès dynamique : structure hybride proposant à la fois des primitives d'accès séquentiel et d'indexation directe.
- Incompatibilité structurelle : interdiction absolue d'appliquer un modèle d'accès direct sur un flux purement séquentiel.

Cas concret enseignant : *Le professeur insiste lourdement sur un piège récurrent en évaluation : rédiger un algorithme qui utilise un accès par indice 'T[i]' pour traiter un fichier texte ou un fichier CSV brut. Comme le système d'exploitation ne fournit pour ces fichiers qu'un curseur séquentiel avance-ligne, l'algorithme est sémantiquement faux et conduit directement à la note zéro sur la question.*

2. Premier Modèle : Le Parcours Complet avec Traitement Systématique

Ce modèle s'applique lorsque chaque élément d'une zone définie d'une collection doit subir une opération uniforme sans aucune exception ni filtrage (exemples : initialiser toutes les cases d'un tableau à zéro, afficher le contenu complet d'un tableau, ou convertir l'ensemble des caractères d'une chaîne en majuscules). Comme la taille du domaine à parcourir est fixée dès le départ, la structure itérative appropriée est la boucle bornée 'Pour'. En notation standard, pour un tableau de N cases, la première se situe à l'indice 0 et la dernière à l'indice N - 1. Le traitement est exécuté de manière indifférenciée et systématique à chaque tour de boucle.

- Objectif : appliquer un traitement uniforme et inconditionnel à 100% des éléments situés entre une borne de début et une borne de fin.
- Structure itérative : utilisation obligatoire d'une boucle bornée 'Pour' (car le nombre d'itérations est calculable à l'avance : Fin - Début + 1).

• Gestion stricte des bornes : pour un tableau de N éléments indicé à partir de 0, la variable d'indice varie strictement de 0 à N - 1.

• Erreur critique Out of Bounds : boucler jusqu'à N entraîne une tentative d'accès hors mémoire.

Cas concret enseignant : *Exemple d'illustration vu en amphithéâtre : 'Afficher l'intégralité d'un tableau de N cases'. L'enseignant rappelle que l'erreur la plus fréquente chez les étudiants débutants consiste à écrire 'Pour i allant de 0 à N', ce qui provoque un crash immédiat ('out of bounds') lors de la tentative de lecture de la case mémoire T[N] qui n'existe pas.*

3. Second Modèle : Le Parcours Complet avec Traitement Conditionné

Le parcours complet avec traitement conditionné conserve l'exploration exhaustive du modèle 1, mais encadre l'action de traitement par une condition de filtrage. Il est utilisé pour compter des occurrences particulières (ex: compter le nombre de voyelles dans un texte), afficher uniquement un sous-ensemble de données (ex: imprimer les nombres pairs d'un tableau), ou effectuer des calculs cumulatifs ciblés. Si la condition compare un élément à son voisin précédent (T[i] > T[i-1]), le concepteur doit modifier la borne initiale (démarrer à i = 1 au lieu de 0) sous peine d'évaluer T[-1] qui provoque une erreur mémoire.

• Parcours exhaustif : tous les éléments de la zone délimitée sont inspectés de la borne initiale à la borne finale.

• Traitement sélectif : l'action principale n'est déclenchée que si l'élément inspecté valide un prédicat logique (test booléen).

• Architecture algorithmique : imbrication d'une structure alternative 'Si ... Alors' au sein d'une boucle bornée 'Pour'.

• Ajustement des bornes : si la condition compare l'élément courant à ses voisins (i-1 ou i+1), les bornes de la boucle doivent être décalées pour éviter tout déréférencement hors limites.

Cas concret enseignant : *L'analogie de la chaîne de montage en usine donnée par le professeur : Sur une chaîne de fabrication, chaque pièce défile obligatoirement devant l'inspecteur qualité (parcours complet d'inspection de toutes les pièces), mais l'inspecteur n'appose le coup de tampon de rebut (action de traitement) que si et seulement si la pièce présente un défaut spécifique (traitement conditionné).*

4. Troisième Modèle : La Recherche de Première Occurrence & Ses Variantes

À l'inverse des parcours complets, la recherche de première occurrence s'interrompt dès que le premier élément satisfaisant le critère est localisé. L'esprit de conception repose sur le principe du pessimisme : on initialise un booléen 'trouve' à 'Faux' avant d'engager l'exploration. Tout algorithme de recherche doit obligatoirement intégrer deux conditions d'arrêt : (1) le succès de la recherche, (2) l'échec par épuisement de la collection. La variante 1 utilise une sortie anticipée (idéale pour récupérer l'indice exact), tandis que la variante 2 utilise une boucle 'Tant que' globale en tête de boucle.

• Pessimisme algorithmique : postulat de départ supposant que la cible est absente de la collection (trouve <- Faux).

• Itération non bornée : emploi obligatoire d'une boucle 'Tant que' (While) car le nombre d'itérations est imprévisible à l'avance.

• Double condition d'arrêt obligatoire : un algorithme de recherche valide s'arrête SI l'élément est trouvé OU SI la collection est entièrement parcourue.

• Variante 1 (Rupture immédiate) : interruption nette dès le succès ; laisse l'indice 'i' positionné exactement sur l'élément trouvé.

• Variante 2 (Tant que global) : condition unique en tête de boucle ('Tant que i < N et NON trouve') ; très propre syntaxiquement mais effectue une incrémentation ultime de 'i' avant de sortir.

Cas concret enseignant : *La règle d'or du professeur : 'En informatique, soyez pessimistes ! Partez toujours du principe que ce que vous cherchez n'existe pas dans la structure, et ne basculez votre indicateur à Vrai qu'au moment précis où vous le touchez.' Il martèle qu'un algorithme de recherche présenté en examen avec une seule condition d'arrêt recevra systématiquement la note zéro.*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Est-ce qu'on peut appliquer le modèle du parcours complet avec traitement systématique sur une structure à accès dynamique ?
Réponse du professeur : Oui, tout à fait. Une structure à accès dynamique propose à la fois des mécanismes d'accès séquentiels et des mécanismes d'accès direct. Par conséquent, il est tout à fait possible d'exploiter les primitives d'accès direct de cette structure pour appliquer un parcours systématique.

Question étudiant : Pourquoi s'embêter à nous enseigner deux variantes différentes pour la recherche de première occurrence ?
Réponse du professeur : Parce que le choix dépend de votre objectif final. La variante 1 (rupture immédiate) est indispensable si vous avez besoin de récupérer la position indicée exacte de l'élément (car l'indice pointe pile sur la cible). La variante 2 (Tant que global) offre une syntaxe de codage beaucoup plus propre et standard avec un 'While' direct, idéale si vous cherchez seulement une simple validation booléenne d'existence.

Question étudiant : Comment fait-on si la condition dans un parcours complet conditionnel dépend de l'élément précédent (par exemple T[i] > T[i-1]) ?
Réponse du professeur : Il faut impérativement réajuster les bornes de votre boucle 'Pour'. Si vous manipulez l'élément i-1, vous ne pouvez pas démarrer votre boucle à i = 0 sous peine de tenter d'évaluer T[-1] (ce qui provoque une erreur de dépassement mémoire). Il faut donc faire démarrer l'indice i à 1.

Question étudiant : Si l'on veut récupérer TOUS les indices correspondant à une valeur donnée (par exemple trouver tous les '5' d'un tableau), quel modèle doit-on utiliser ?
Réponse du professeur : Il ne faut pas utiliser un modèle de recherche de première occurrence (qui s'arrête au premier trouvé), mais un 'Parcours complet avec traitement conditionné', où la condition teste si l'élément vaut 5 et où le traitement consiste à stocker ou afficher l'indice i.

■ Exercice & Démonstration d'Application

Énoncé : Écrire en pseudo-code algorithmique rigoureux l'algorithme permettant d'afficher tous les nombres strictement positifs d'un tableau d'entiers 'T' contenant 'N' éléments. Expliciter la démarche, le modèle retenu et la gestion des bornes.

Résolution pas à pas :
Étape 1 : Analyse du problème et sélection du modèle. Il faut inspecter l'intégralité des cases d'un tableau (structure à accès direct) de la première à la dernière. L'action d'affichage ne doit s'exécuter que si la valeur respecte le critère 'strictement supérieur à 0'. Le modèle algorithmique approprié est donc le 'Parcours complet avec traitement conditionné'. Étape 2 : Détermination des bornes d'indices. En convention informatique standard (indexation à zéro), pour un tableau de N éléments, la première case est à l'indice 0 et la dernière à l'indice N - 1. Borne initiale = 0, Borne finale = N - 1. Étape 3 : Choix de la structure de contrôle. Le nombre d'itérations est connu avant d'entrer dans la boucle -> Utilisation d'une boucle bornée 'Pour i allant de 0 à N - 1'. Étape 4 : Écriture du filtre conditionnel et du traitement. À chaque itération i, évaluer 'Si T[i] > 0 Alors' puis déclencher l'action 'Afficher(T[i])'.

Résultat : Algorithme AffichagePositifs Données T : Tableau d'entiers N : Entier (taille du tableau) Variables i : Entier Début Pour i allant de 0 à N - 1 Faire Si T[i] > 0 Alors Afficher(T[i]) Fin Si Fin Pour Fin Commentaire didactique : Cet algorithme garantit un parcours exhaustif sans risque de dépassement de capacité (l'itération s'arrête exactement à N-1) et applique le filtre sélectif demandé de manière optimale.

■ Auto-Évaluation Flash d'Examen

Q : Quel est le risque critique si l'on écrit une boucle 'Pour i allant de 0 à N' sur un tableau de N éléments indicé à partir de 0 ?

R : Une erreur fatale de dépassement d'indice mémoire ('out of bounds'), car le tableau possède des indices de 0 à N-1.

Q : Combien de conditions d'arrêt minimales doit comporter la structure itérative d'une recherche de première occurrence ?

R : Exactement deux conditions d'arrêt : (1) Le succès (l'élément est trouvé) et (2) L'échec (la structure a été entièrement parcourue).

Q : Pourquoi qualifie-t-on la démarche d'initialisation d'un algorithme de recherche de première occurrence de 'pessimiste' ?

R : Parce que l'on initialise systématiquement l'indicateur booléen 'trouvé' à 'Faux', en posant comme postulat de départ que la cible n'existe pas dans la collection.